HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO HÀNG TUẦN

Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng GraphRAG

Lớp: E22HTTT

Tên: Nguyễn Đức Anh

ID: B22DCDT017

Người hướng dẫn: Kim Ngọc Bách

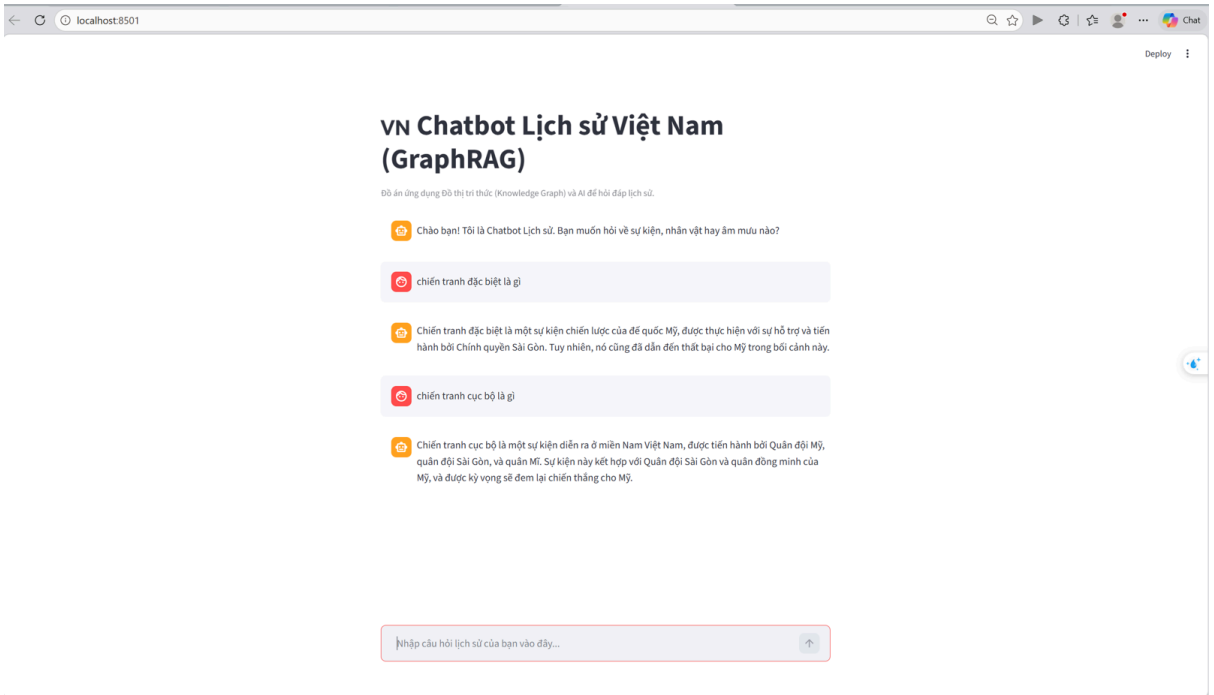
Hà Nội – 2026

Weekly Report 9

I. OVERVIEW OF PROGRESS IN THE PAST WEEK

After completing the core data processing pipeline (Backend) to extract and store the Knowledge Graph on Neo4j, the focus of this week was on building the User Interface (Frontend) and optimizing the interactive experience.

I successfully transitioned the application from a Command Line Interface (CLI) to a fully functional Web Application using Python/Streamlit, seamlessly integrating it with the GraphRAG system.



II. DETAILS OF COMPLETED TASKS

1. Building the Web Application Frontend

- Streamlit Framework Integration: Successfully initialized and configured the Chatbot's graphical interface, complete with a title, icon, and visual description (st.set_page_config, st.title).
- Conversational Environment: Built a communication flow that simulates modern messaging apps, utilizing st.chat_message and st.chat_input to clearly separate the student's input from the AI's responses.
- Waiting Effects Integration (UX): Added a loading indicator (st.spinner("Searching the Neo4j Knowledge Graph...")) while the system

generates Cypher queries and retrieves data from the database, significantly improving the overall User Experience (UX).

2. State and Conversation History Management (Session Management)

- Session State Implementation: Programmed the st.session_state.messages mechanism, granting the Chatbot the ability to store and display the entire chat history during an active session.
- Greeting Initialization: Configured the system to automatically send a welcome message and suggest topics (e.g., "events, figures, or plots") as soon as a user accesses the platform.

3. Performance & Connection Optimization (Caching)

- Caching Mechanism Application (@st.cache_resource): This is a crucial technical improvement. The initialization of the Neo4j database connection and the loading of the LLM (gpt-4o-mini) are now cached. This prevents the system from reconnecting from scratch every time a user sends a new message (due to Streamlit's re-run mechanism), thereby minimizing response latency.
- Environment Variable Control: Restructured the code to force the system to load the .env file (containing secure API Keys and database credentials) from the very first lines, ensuring stable and secure database connections in the Web environment.

4. Inheriting and Consolidating the GraphRAG Backend

- Maintaining Smart Prompt Engineering: Successfully integrated the optimized CYPHER_GENERATION_TEMPLATE from the previous week (which supports Fuzzy Search and multi-dimensional semantic queries) directly into the Web App's execution flow.
- Web-based Error Handling: Added try...except blocks combined with st.error() to automatically catch errors and display user-friendly notifications (e.g., if the database connection drops or the AI finds no relevant graph data), preventing the application from crashing.

III. PLANS AND FUTURE DEVELOPMENT DIRECTIONS

- Vector Database Integration: Combine Vector/Embeddings search for long-form content (detailed event descriptions) in parallel with Graph search to provide richer system responses.

- GraphRAG Upgrade (Multi-hop QA): Further fine-tune the system so the Chatbot can handle complex queries requiring 2-3 hops across the graph (e.g., "Who was the commander in the battle where weapon X was used?").